

PROJECT REPORT

ATTENDANCE CALCULATOR

Project Type	Console-Based Application
Programming Language	Python
Subject	Introduction to Problem Solving
Name	Abhay Pal
Registration Number	26BCE10249
Academic Year	2026–27

2. Introduction

Attendance is an important part of academic management because students need to maintain a minimum percentage of attendance to remain eligible for classes and academic activities.

The **Attendance Calculator** is a simple console-based Python application developed to calculate a student's attendance percentage based on the total number of classes conducted and the number of classes attended.

The program takes the student's name, subject name, total number of classes conducted, and classes attended as input. It then calculates the attendance percentage and displays a corresponding remark and eligibility status.

The application also provides an additional feature for students whose attendance is below 75%. It calculates the number of consecutive classes they need to attend to reach the 75% attendance requirement.

3. Problem Statement

Students often need to manually calculate their attendance percentage and determine whether they meet the required attendance criteria. Manual calculations can be inconvenient and may lead to calculation errors.

The objective of this project is to develop a simple Python-based application that:

- Accepts student attendance details as input.
- Calculates attendance percentage automatically.
- Validates the entered attendance information.
- Displays a suitable remark based on attendance percentage.
- Determines whether the student is eligible or not eligible.
- Calculates the number of consecutive classes required to reach 75% attendance when the current attendance is below 75%.

4. Functional Requirements

Requirement	Description
Student Information Input	Accept student name and subject name.
Attendance Input	Accept total classes conducted and classes attended.
Input Validation	Check that total classes are greater than 0 and attended classes are within the valid range.
Attendance Calculation	Calculate attendance using attended classes divided by total classes, multiplied by 100.
Remark Generation	Display a remark based on the calculated attendance percentage.
Eligibility Status	Display Eligible for attendance of 75% or above; otherwise Not Eligible.
Required Classes Calculation	When attendance is below 75%, calculate the consecutive classes needed to reach 75%.

5. Non-functional Requirements

- **Simplicity:** The application should be simple and easy for a beginner to understand and use.
- **Usability:** The program should provide clear prompts and readable output.
- **Accuracy:** Attendance percentage and required-classes calculations should produce accurate results based on valid input.

- **Reliability:** The program should handle invalid attendance values using conditional checks before calculations.
- **Performance:** The application performs basic calculations and therefore requires very little processing time.
- **Maintainability:** The use of simple variables, conditional statements, and a loop keeps the program relatively easy to understand and modify.

6. System Architecture

The Attendance Calculator follows a simple **input–process–output architecture**.

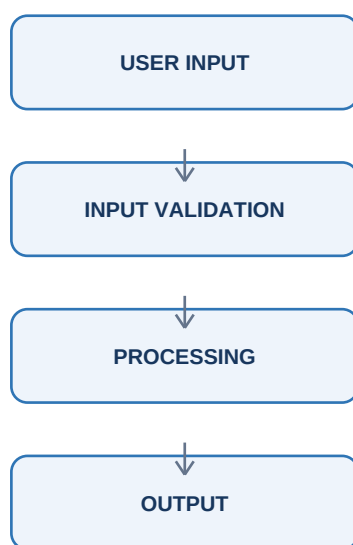


Figure 1. High-level system architecture

- **Input Layer:** Collects student name, subject name, total classes, and attended classes.
- **Validation Layer:** Checks whether the entered attendance values are valid.
- **Processing Layer:** Calculates attendance and determines the appropriate remark and eligibility status.
- **Additional Calculation:** Uses a while loop to determine the classes required to reach 75% when attendance is below 75%.
- **Output Layer:** Displays the final student and attendance information.

7. Design Diagrams

7.1 Use Case Diagram

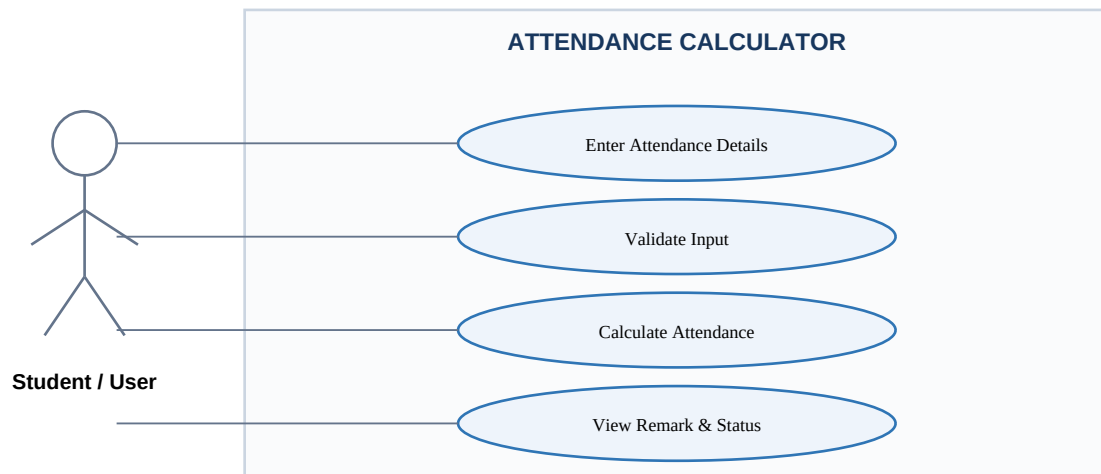


Figure 2. Use case diagram

7.2 Workflow Diagram

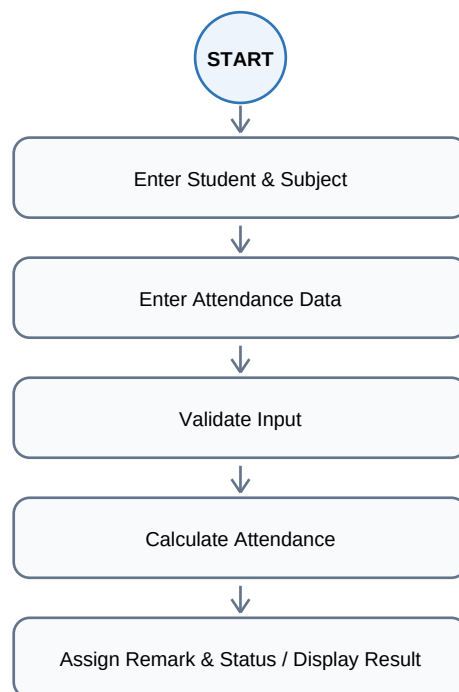


Figure 3. Workflow diagram

7.3 Sequence Diagram

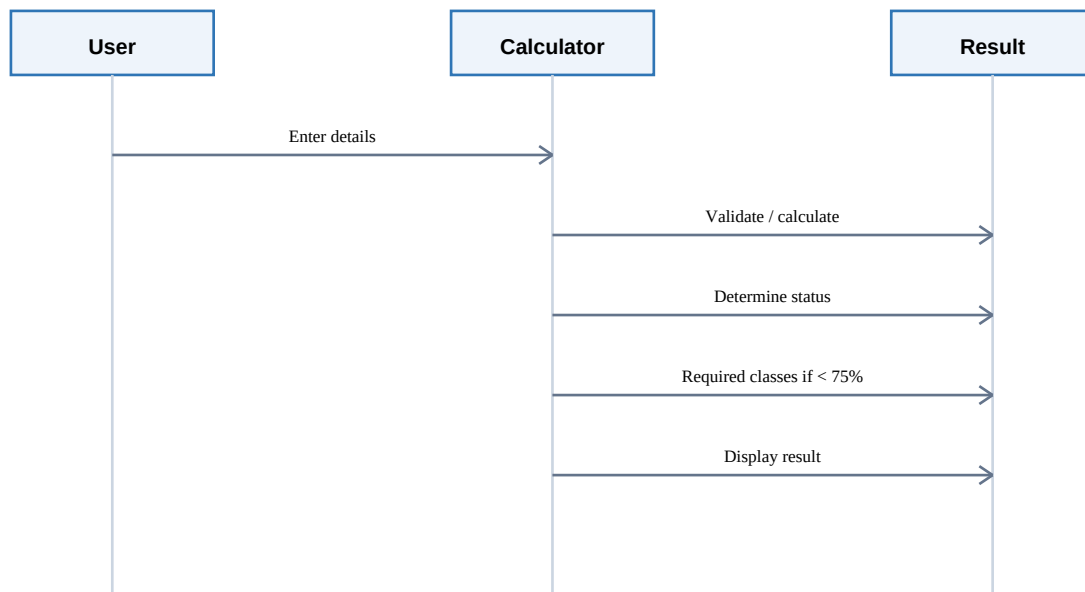


Figure 4. Sequence diagram

7.4 Class / Component Diagram

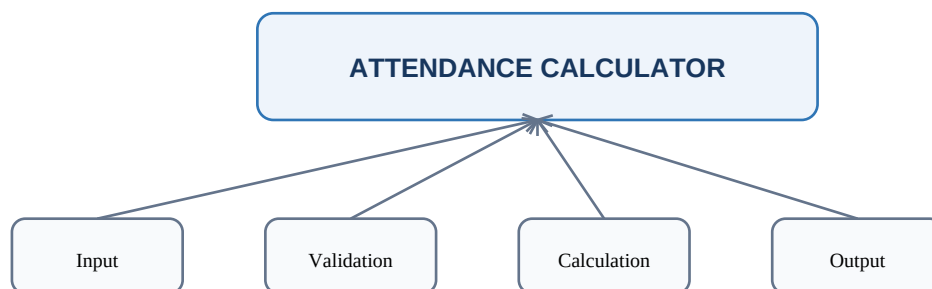


Figure 5. Component view. A traditional class diagram is not applicable because the program is procedural and defines no classes.

7.5 ER Diagram

Not Applicable. The application does not use a database or permanent data storage. User input exists only during program execution, so there are no persistent entities or relationships to represent.

8. Design Decisions & Rationale

Decision	Rationale
Console-Based Interface	Selected because the project focuses on Python programming fundamentals rather than graphical interface development.
Conditional Statements	if, elif, and else are used for validation, attendance categorization, and eligibility decisions.
While Loop	Used to repeatedly test future attendance until the calculated percentage reaches at least 75%.
No Database	Permanent storage is outside the current scope because the application performs immediate calculations and displays the result.

Decision	Rationale
Simple Variables	Basic variables such as name, sub, total, a, attendance, and c keep the implementation straightforward.

9. Implementation Details

The project is implemented using **Python** and follows a procedural programming approach.

Input: The program uses input() to collect student name, subject name, total classes, and attended classes. Total classes and attended classes are converted to integers using int() because they represent whole-number counts.

Validation: The program checks whether total classes are greater than zero and whether attended classes are within the range from zero to total classes.

Attendance Calculation: $\text{Attendance} = (\text{Classes Attended} / \text{Total Classes}) \times 100$.

Attendance	Remark	Status
$\geq 90\%$	Excellent	Eligible
$\geq 85\%$ and $< 90\%$	Good	Eligible
$\geq 75\%$ and $< 85\%$	Satisfactory-can be improved	Eligible
$\geq 65\%$ and $< 75\%$	Low Attendance	Not Eligible
$< 65\%$	Very Low Attendance	Not Eligible

Required Classes Calculation: When attendance is below 75%, c starts from zero. A while loop repeatedly evaluates $((a + c) / (\text{total} + c)) \times 100 < 75$. The value of c is increased until the attendance reaches at least 75%.

10. Screenshots / Results

The report can include execution screenshots from the submitted program. Representative scenarios are listed below.

Scenario	Example Input	Expected Result
Eligible student	Total = 40; Attended = 38	95%; Excellent; Eligible
Below 75%	Total = 40; Attended = 25	62.5%; Very Low Attendance; Not Eligible; required classes shown
Invalid total	Total = 0	"Total classes should be greater than 0."

11. Testing Approach

Test Case	Input Condition	Expected Result
TC01	Total classes = 0	Error message displayed
TC02	Attended classes < 0	Invalid attendance message
TC03	Attended classes $>$ total classes	Invalid attendance message
TC04	Attendance $\geq 90\%$	Excellent, Eligible
TC05	Attendance 85% to $< 90\%$	Good, Eligible
TC06	Attendance 75% to $< 85\%$	Satisfactory-can be improved, Eligible

Test Case	Input Condition	Expected Result
TC07	Attendance 65% to < 75%	Low Attendance, Not Eligible
TC08	Attendance < 65%	Very Low Attendance, Not Eligible
TC09	Attendance < 75%	Required classes calculation performed
TC10	Attendance exactly 75%	Eligible; required-class calculation not performed

12. Challenges Faced

- **Handling Invalid Input:** Ensuring attended classes cannot be negative or greater than total classes.
- **Attendance Categories:** Arranging conditions correctly so each percentage falls into the intended category.
- **Required Classes Calculation:** Using repeated calculations through a while loop to determine how many future classes are required to reach 75%.
- **Avoiding Incorrect Calculations:** Validating input before calculating attendance prevents meaningless results from invalid values.

13. Learnings & Key Takeaways

- Taking user input using `input()`.
- Converting input to integers using `int()`.
- Using variables to store information.
- Performing arithmetic calculations.
- Using `if`, `elif`, and `else` for decision making.
- Applying logical operators such as `or`.
- Using a while loop for repeated calculations.
- Validating user input.
- Designing a simple console-based Python application.
- Converting a real-world problem into programming logic.

The project demonstrates how fundamental programming concepts can be combined to create a small but functional application.

14. Future Enhancements

- **Multiple Subject Support:** Allow attendance details for multiple subjects.
- **Data Storage:** Store attendance records using files or a database.
- **Graphical User Interface:** Add a GUI for easier interaction.
- **Attendance History:** Maintain previous attendance records.
- **Additional Attendance Calculations:** Calculate how many classes can be missed while still maintaining 75% attendance.
- **Student Login System:** Allow individual students to access their attendance information.
- **Report Generation:** Generate a detailed attendance report.

15. References

- Python Software Foundation — Python programming language documentation.
- Python documentation covering input, conditional statements, arithmetic operations, and loops.
- Course notes and classroom material related to Python programming fundamentals.
- Project source code developed for the Attendance Calculator.